GRACE: A GraphRAG Causal Benchmark

Welcome to the **GRACE** benchmark repository. This project aims to evaluate the robustness and causal reasoning capabilities of various GraphRAG (Retrieval-Augmented Generation over Knowledge Graphs) frameworks when faced with different graph perturbations.

Installation & Setup

This project uses modern Python dependency management via `uv`.

```
# Cài đặt nền tảng và môi trường ảo
uv sync
```

Configuration (Environment Variables)

Before running the benchmark, you must configure the LLM endpoints. We use a `.env` file for easy management.

```
# Copy the provided template
cp .env.template .env
```

Open the `.env` file and insert your configuration for local LLM and remote Embedding API:

- **OPENAI_API_KEY**: Required for Nvidia Embeddings.
- **OPENAI_BASE_URL**: Point to Nvidia's API (`https://integrate.api.nvidia.com/v1`).
- **EMBEDDING_BASE_URL**: Point to Nvidia's API (`https://integrate.api.nvidia.com/v1`).
- **EMBEDDING_MODEL_NAME**: Set the embedding model (default: `nvidia/nv-embed-v1`).
- **LLM_BASE_URL**: Point to your local vLLM instance (e.g., `http://localhost:8001/v1`).
- **LLM_MODEL_NAME**: Set your local model name (e.g., `cyankiwi/Qwen3-30B-A3B-Instruct-2507-AWQ-4bit`).

PROF

Running Benchmarks

1. Start Local vLLM Server

We run the 30 tỷ tham số LLM locally (`cyankiwi/Qwen3-30B-A3B-Instruct-2507-AWQ-4bit`) using 4-bit bitsandbytes quantization to fit into a single GPU (e.g., RTX 5880 Ada with 48GB VRAM).

First, install the required serving dependencies if you haven't already:

```
uv pip install vllm "bitsandbytes>=0.48.1"
```

Start the vLLM server on a dedicated Terminal. Dưới đây là giải thích chi tiết cho các cấu hình (config) đang được sử dụng để chạy model này mượt mà:

- **`CUDA_VISIBLE_DEVICES=2`**: Chỉ định phiên chạy trên GPU số 2 (do GPU này đang trống toàn bộ ~48GB VRAM).
- **`VLLM_USE_V1=0`**: Tắt Engine V1 mới của vLLM để dùng lại V0. Engine V1 hiện tại đang lỗi tương thích bộ nhớ với dòng model MoE (Mixture of Experts) to như Qwen.
- **`--quantization compressed-tensors`**: Quá trình nhồi model vào RAM. Model do cộng đồng đóng gói được định dạng dưới chuẩn `compressed-tensors`, nên vLLM cần cấu hình này để phân giải model 4-bit thành công.
- **`--max-model-len 16384`**: Mở rộng độ dài Context Window từ 8K lên 16K tokens. Điều này cực kỳ quan trọng để đảm bảo thuật toán Graph Retrieval của LightRAG không bị tràn context (trả về lỗi `None`) khi truy xuất các Subgraphs quá lớn và có tiểu sử dài.
- **`--gpu-memory-utilization 0.95`**: Cho quyền vLLM được chiếm đến 95% thẻ GPU để tăng tỉ lệ cache nội bộ KV.
- **`--enforce-eager`**: Ép buộc chạy dạng Eager thay cho CUDA Graph. Với model MoE, CUDA Graph hay gặp lỗi fragmentation (kẹt phân mảnh bộ nhớ) gây dừng startup.

```
CUDA_VISIBLE_DEVICES=0 VLLM_USE_V1=0 uv run python -m
vllm.entrypoints.openai.api_server \
    --model cyankiwi/Qwen3-30B-A3B-Instruct-2507-AWQ-4bit \
    --quantization compressed-tensors \
    --host 0.0.0.0 \
    --port 8001 \
    --max-model-len 16384 \
    --gpu-memory-utilization 0.9 \
    --enforce-eager
```

Note: Loading and quantizing a 60GB+ 30B model on-the-fly may take 10-15 minutes. Wait until you see `Uvicorn running on http://0.0.0.0:8001` before proceeding.

PROF

2. Data Preparation (Wikidata Context)

Before evaluating, you must fetch the metadata (labels, descriptions, aliases) for all Wikidata entities and properties present in the subgraphs. This process ensures that the RAG models understand the knowledge triples natively in English.

```
uv run python scripts/data_generation/fetch_wikidata_labels.py
```

This cache is stored in `data/wikidata_labels.json` and maps graph IDs to rich contextual text.

3. Centralized Prompts

To ensure fair evaluation across all RAG frameworks, we use a single, strict benchmark instruction. This guarantees that models output **pure entity names** and handle dataset anomalies correctly (e.g., avoiding

literal numbers when the dataset ground truth expects an entity, or isolated/broken graphs).

This prompt is located at: `src/graphrag_benchmark/prompts.py`

4. LightRAG Benchmark Execution

After your local vLLM server is running and data is collected, open a **new Terminal** to run the complete causal benchmark evaluation script.

Run across all subgraphs:

```
uv run scripts/benchmark/run_all_lightrag.py
```

Run a single query by ID:

```
uv run python scripts/benchmark/benchmark_lightrag.py 2466
```

Note: The script will load a subgraph scenario from `.json` files, construct Knowledge Graphs dynamically using the RAG engine for all available variants (`clean`, `broken`, `type_matching`, `topological`, `swapping`), query the local Qwen LLM while fetching Embeddings from Nvidia, and then score the responses against the Ground Truth.

Data Extraction & Analysis

The data pipeline scripts have been logically classified within the `scripts/` folder:

- **`scripts/data_generation/`**: Utilities for extracting and perturbing source data (LC-QuAD 2.0).
- **`scripts/analysis/`**: Diagnostic utilities to verify metadata, logs, and diagram visualization.

PROF

If you want to learn how the datasets are exactly extracted from Wikidata, how causal perturbations are applied, and how they are statistically analyzed or visualized, please read our detailed methodology guide in:

☞ **README_DATASET.md**